EXCEL LEARNING HANDBOOK

Text Functions

Lessons 59–67 | Complete Notes, Syntax, Worked Examples and Expected Results

LEFT • RIGHT • MID • UPPER • LOWER • PROPER • LEN • TRIM • CLEAN • FIND • SEARCH • SUBSTITUTE • REPLACE • CONCAT • TEXTJOIN • TEXTBEFORE • TEXTAFTER • TEXTSPLIT • TEXT • TRANSLATE • DETECTLANGUAGE

Scope: This handbook consolidates the text-function lessons completed in our Excel course. It includes explanations, formulas, demonstrations, result tables, practical notes and troubleshooting. Practice assignments are intentionally excluded.

Excel versions: Classic text functions work widely. TEXTBEFORE, TEXTAFTER and TEXTSPLIT require newer Excel editions. TRANSLATE and DETECTLANGUAGE are Microsoft 365 features and use translation services. Function separators and output language may depend on regional settings.

Example notation: Formulas shown with commas use English-language Excel formula syntax. Sample rows are illustrative; expected results correspond to the stated values.

Contents and Learning Map

Lesson	Topic	Main outcome
59	LEFT, RIGHT & MID	Extract characters by position
60	UPPER, LOWER & PROPER	Standardize capitalization
61	LEN, TRIM & CLEAN	Count characters and clean imported text
62	FIND & SEARCH	Locate text and extract dynamically
63	SUBSTITUTE & REPLACE	Replace exact text or positioned characters
64	CONCAT & TEXTJOIN	Combine text with optional separators
65	TEXTBEFORE, TEXTAFTER & TEXTSPLIT	Extract around delimiters and split into arrays
66	TEXT	Format numbers, dates, percentages and time as text
67	TRANSLATE & DETECTLANGUAGE	Translate between languages and detect source language

What is included in each lesson?

Definition and use cases; syntax and argument meanings; worksheet context; worked formulas with verified expected outputs; important behavior; mistakes and limitations. A comparative cheat sheet and worked business scenarios appear at the end.

LESSON 59 | LEFT, RIGHT & MID

Use these functions to extract parts of strings. LEFT starts at the beginning, RIGHT starts at the end, and MID begins at a specified character position. Excel positions start at 1, and spaces count as characters.

Syntax and arguments

LEFT: Return first N characters; N defaults to 1.

```
=LEFT(text,[num_chars])
```

RIGHT: Return last N characters; N defaults to 1.

```
=RIGHT(text,[num_chars])
```

MID: Return a chosen number of characters starting at a 1-based position.

```
=MID(text,start_num,num_chars)
```

Example data / worksheet setup

Use example text: A2 = LV-Louis Vuitton; B2 = Louis Vuitton; C2 = PRD-2026-1001.

Worked examples and expected outputs

1. Extract first three letters

```
=LEFT("Louis Vuitton",3)
```

Expected result: Lou. The first three characters are L, o and u.

2. Extract first word of fixed length

```
=LEFT("Louis Vuitton",5)
```

Expected result: Louis. Five characters from the left.

3. Extract final three characters

```
=RIGHT("Louis Vuitton",3)
```

Expected result: ton. The last three letters of Vuitton.

4. Extract 4-digit product serial

```
=RIGHT("PRD-2026-1001",4)
```

Expected result: 1001. The result is text, so leading zeros would be preserved.

5. Extract three letters from middle

```
=MID("LV-Louis Vuitton",4,3)
```

Expected result: Lou. Starts at fourth character and returns three.

6. Extract year from product ID

```
=MID("PRD-2026-1001",5,4)
```

Expected result: 2026. Characters 5 through 8 contain the year.

7. Convert extracted digits into a number

```
=VALUE(MID("PRD-2026-1001",5,4))
```

Expected result: 2026 (number). VALUE converts numeric-looking text into a numeric value.

Important concepts and best practices

• Character counts include spaces and punctuation.

• If requested count exceeds available length, return available text.

• MID with start position beyond the end returns empty text.

• All three extraction functions return text, including when the characters are digits.

Common mistakes / limitations

• Do not count from zero: Excel uses position 1.

• For variable-length labels, use FIND/SEARCH or TEXTBEFORE instead of hard-coded positions.

• Use VALUE only when numeric calculation is needed.

LESSON 60 | UPPER, LOWER & PROPER

These functions normalize capitalization without changing source cells. UPPER makes all letters uppercase; LOWER makes them lowercase; PROPER capitalizes the first letter after common word boundaries.

Syntax and arguments

UPPER: Convert letters to uppercase.

```
=UPPER(text)
```

LOWER: Convert letters to lowercase.

```
=LOWER(text)
```

PROPER: Convert letters into title-like capitalization.

```
=PROPER(text)
```

Example data / worksheet setup

Example input B2:B9: louis vuitton; CHANEL; gUcCi; NIKE; zara; H&M; calvin klein; tOMMY hILFIGER.

Worked examples and expected outputs

1. All uppercase

```
=UPPER("gUcCi")
```

Expected result: GUCCI. Letter case is standardized.

2. All lowercase

```
=LOWER("CHANEL")
```

Expected result: chanel. Every uppercase letter is changed.

3. Proper case

```
=PROPER("tOMMY hILFIGER")
```

Expected result: Tommy Hilfiger. Both words get a capital initial.

4. Customer name standardization

```
=PROPER("rAVI kUMAR")
```

Expected result: Ravi Kumar. Useful for inconsistent names.

5. Extract and capitalize prefix

```
=UPPER(LEFT("louis vuitton",3))
```

Expected result: LOU. Nested functions first extract and then capitalize.

6. Brand-specific capitalization caveat

```
=PROPER("iPHONE")
```

Expected result: Iphone. PROPER does not understand brand styling.

Important concepts and best practices

• UPPER, LOWER and PROPER accept a single text argument.

• Numbers, spaces and common symbols are retained.

• Use formula results in helper columns; copy and Paste Values when you need finalized data.

Common mistakes / limitations

• PROPER is not guaranteed to preserve names such as McDonald or iPhone.

• These functions do not remove extra spaces; use TRIM separately.

• Changing capitalization is a transformation of returned text, not merely font styling.

LESSON 61 | LEN, TRIM & CLEAN

Use LEN to measure text length, TRIM to remove unnecessary normal spaces, and CLEAN to remove certain non-printable ASCII control characters. Together these functions address common problems in imported datasets.

Syntax and arguments

LEN: Count characters, including ordinary spaces.

```
=LEN(text)
```

TRIM: Remove leading/trailing regular spaces and reduce internal runs to one.

```
=TRIM(text)
```

CLEAN: Remove ASCII non-printing characters 0 to 31.

```
=CLEAN(text)
```

Example data / worksheet setup

Example input: B2 = Louis Vuitton; B3 = " Chanel "; B4 = "Gucci Fashion". Quoted spaces are intentional.

Worked examples and expected outputs

1. Count characters

```
=LEN("Louis Vuitton")
```

Expected result: 13. Five letters, a space and seven letters.

2. Trim leading/trailing spaces

```
=TRIM(" Chanel ")
```

Expected result: Chanel. Removes four unnecessary outer spaces.

3. Trim repeated internal spaces

```
=TRIM("Gucci Fashion")
```

Expected result: Gucci Fashion. Three internal spaces become one.

4. Find number of characters removed

```
=LEN(" Chanel ")-LEN(TRIM(" Chanel "))
```

Expected result: 4. Original length 10 minus cleaned length 6.

5. Remove a line feed

```
=CLEAN("Hello"&CHAR(10)&"World")
```

Expected result: HelloWorld. CLEAN removes line feed but does not insert a space.

6. Clean then trim

```
=TRIM(CLEAN(" Hello"&CHAR(10)&" World "))
```

Expected result: Hello World. The existing space before World separates the words.

7. Handle nonbreaking spaces

```
=TRIM(SUBSTITUTE(A2,CHAR(160)," "))
```

Expected result: cleaned text. Replace nonbreaking spaces with standard spaces before TRIM.

Important concepts and best practices

• LEN returns a number; TRIM and CLEAN return text.

• Normal spaces are ASCII 32, while nonbreaking spaces are often CHAR(160).

• CLEAN does not remove every possible Unicode control character.

• To measure trimmed length use LEN(TRIM(A2)).

Common mistakes / limitations

• TRIM does not remove all unusual whitespace characters.

• CLEAN removes line-feed/tab characters rather than replacing them with spaces; words may become joined.

• Use a backup and Paste Values only after validating cleaned results.

LESSON 62 | FIND & SEARCH

FIND and SEARCH return the 1-based starting position of matching text. FIND is case-sensitive; SEARCH ignores case and supports wildcard patterns. Use them with LEFT, RIGHT, MID and LEN for variable-length extraction.

Syntax and arguments

FIND: Locate exact-case text.

```
=FIND(find_text,within_text,[start_num])
```

SEARCH: Locate text without case sensitivity; supports ? and * wildcards.

```
=SEARCH(find_text,within_text,[start_num])
```

Example data / worksheet setup

Example source A2 = EMP_2026_1045; B2 = john.doe@example.com; D2 = Excel Training.

Worked examples and expected outputs

1. Locate at-sign

```
=FIND("@","john.doe@example.com")
```

Expected result: 9. The @ occurs at position 9.

2. Locate first underscore

```
=FIND("_","EMP_2026_1045")
```

Expected result: 4. First underscore occurs after EMP.

3. Locate second underscore

```
=FIND("_","EMP_2026_1045",5)
```

Expected result: 9. Search begins after the first underscore.

4. Find second underscore dynamically

```
=FIND("_",A2,FIND("_",A2)+1)
```

Expected result: 9 if A2 = EMP_2026_1045. Nested FIND computes the next start position.

5. Find word start

```
=FIND("Training","Excel Training")
```

Expected result: 7. Training begins at position 7.

6. Case difference

```
=SEARCH("excel","Excel Training")
```

Expected result: 1. SEARCH ignores case.

7. Case-sensitive missing text

```
=FIND("excel","Excel Training")
```

Expected result: #VALUE!. FIND does not match capital E to lowercase e.

8. Extract username

```
=LEFT(B2,FIND("@",B2)-1)
```

Expected result: john.doe if B2 = john.doe@example.com. FIND supplies the dynamic character count.

9. Extract email domain

```
=RIGHT(B2,LEN(B2)-FIND("@",B2))
```

Expected result: example.com. RIGHT takes characters after @.

Important concepts and best practices

• If no match exists, FIND and SEARCH return #VALUE!.

• An optional start_num searches at or after that position; indexing begins at 1.

• FIND and SEARCH return positions, not extracted words.

• SEARCH supports wildcards * and ?, while FIND treats them literally.

Common mistakes / limitations

• Use IFERROR(FIND(...),"Not Found") for readable missing-match results.

• Do not confuse start_num with the number of characters to extract.

• Repeated matches require a suitable starting position or nesting.

LESSON 63 | SUBSTITUTE & REPLACE

SUBSTITUTE replaces matching text, whereas REPLACE changes characters based on their positions. SUBSTITUTE is best for identifiable characters and words; REPLACE is convenient for fixed-layout codes.

Syntax and arguments

SUBSTITUTE: Replace all instances or a chosen occurrence of exact text.

```
=SUBSTITUTE(text,old_text,new_text,[instance_num])
```

REPLACE: Replace specified positions in a string.

```
=REPLACE(old_text,start_num,num_chars,new_text)
```

Example data / worksheet setup

Example A2 = EMP-2024-101; B2 = john.doe@example.com.

Worked examples and expected outputs

1. Replace first dot

```
=SUBSTITUTE("john.doe@example.com",".","_",1)
```

Expected result: john_doe@example.com. Only first dot is changed.

2. Replace second dot

```
=SUBSTITUTE("john.doe@example.com",".","_",2)
```

Expected result: john.doe@example_com. Only second dot is changed.

3. Replace every dot

```
=SUBSTITUTE("john.doe@example.com",".","_")
```

Expected result: john_doe@example_com. Omitting occurrence replaces all matches.

4. Replace separators

```
=SUBSTITUTE("EMP-2024-101","-","_")
```

Expected result: EMP_2024_101. Every hyphen becomes an underscore.

5. Replace word

```
=SUBSTITUTE("Monthly Sales Report","Sales","Revenue")
```

Expected result: Monthly Revenue Report. Text match rather than position.

6. Update fixed year

```
=REPLACE("EMP-2024-101",5,4,"2025")
```

Expected result: EMP-2025-101. Position 5 begins four-digit year.

7. Change string length

```
=REPLACE("ABC-123",1,3,"X")
```

Expected result: X-123. Replacement length can differ from removed length.

8. Locate year dynamically

```
=REPLACE(A2,FIND("-",A2)+1,4,"2025")
```

Expected result: USER-2025-101 if A2 = USER-2024-101. FIND identifies position after prefix.

Important concepts and best practices

• SUBSTITUTE is case-sensitive; it does not change text if a match is absent.

• REPLACE is based on positions and does not search for a particular word.

• Changing formula results does not overwrite source cells automatically.

Common mistakes / limitations

• Specify instance_num only when a particular matching occurrence should change.

• REPLACE with a wrong start position can silently corrupt the wrong substring.

• For changing identifiers in place, verify results before Paste Values.

LESSON 64 | CONCAT & TEXTJOIN

CONCAT combines values without separators unless you insert them. TEXTJOIN inserts a chosen delimiter and can skip blanks. The & operator is an additional quick way to combine short strings.

Syntax and arguments

CONCAT: Combine many strings or ranges.

```
=CONCAT(text1,[text2],...)
```

TEXTJOIN: Combine data with separator; TRUE skips empty cells.

```
=TEXTJOIN(delimiter,ignore_empty,text1,[text2],...)
```

Join with &: Combine a few values directly.

```
=A2&" "&B2
```

Example data / worksheet setup

Example row: A2=101; B2=Ravi; C2=Kumar; D2=Sharma; E2=IT; F2=Bengaluru.

Worked examples and expected outputs

1. Concatenate raw names

```
=CONCAT("Ravi","Sharma")
```

Expected result: RaviSharma. CONCAT adds no space automatically.

2. Insert a space

```
=CONCAT("Ravi"," ","Sharma")
```

Expected result: Ravi Sharma. Space is a separate argument.

3. Join three name cells

```
=TEXTJOIN(" ",TRUE,B2:D2)
```

Expected result: Ravi Kumar Sharma if B2:D2 = Ravi,Kumar,Sharma. A delimiter is added between nonblank values.

4. Ignore missing middle name

```
=TEXTJOIN(" ",TRUE,"Priya","","Rao")
```

Expected result: Priya Rao. Empty component is skipped.

5. Retain blank position

```
=TEXTJOIN("-",FALSE,"Priya","","Rao")
```

Expected result: Priya--Rao. Two hyphens surround the empty component.

6. Department-city label

```
=TEXTJOIN(" - ",TRUE,"IT","Bengaluru")
```

Expected result: IT - Bengaluru. Delimiter contains spaces and a hyphen.

7. Create code

```
=CONCAT("IT","-",101)
```

Expected result: IT-101. Number is converted to text during concatenation.

8. Comma-separated list

```
=TEXTJOIN(", ",TRUE,"IT","HR","Finance")
```

Expected result: IT, HR, Finance. TEXTJOIN inserts consistent separators.

Important concepts and best practices

• CONCAT supports range arguments; the older CONCATENATE function is retained for compatibility.

• TEXTJOIN with TRUE ignores genuinely empty entries, not cells containing ordinary spaces.

• Duplicates are not removed by TEXTJOIN.

• TEXTJOIN output is text and has Excel character-length limits.

Common mistakes / limitations

• Do not expect CONCAT to add automatic spaces.

• Set ignore_empty = TRUE where blanks should not create doubled delimiters.

• Clean whitespace and duplicates separately where required.

LESSON 65 | TEXTBEFORE, TEXTAFTER & TEXTSPLIT

These modern Excel functions work with delimiters. TEXTBEFORE and TEXTAFTER return text on one side of a delimiter; TEXTSPLIT returns an array across columns or rows.

Syntax and arguments

TEXTBEFORE: Extract before a chosen delimiter occurrence.

```
=TEXTBEFORE(text,delimiter,[instance_num],[match_mode],[match_end],[if_not_found])
```

TEXTAFTER: Extract after a chosen delimiter occurrence.

```
=TEXTAFTER(text,delimiter,[instance_num],[match_mode],[match_end],[if_not_found])
```

TEXTSPLIT: Split text into spilled columns or rows.

```
=TEXTSPLIT(text,col_delimiter,[row_delimiter],[ignore_empty],[match_mode],[pad_with])
```

Example data / worksheet setup

Example A2 = EMP-2026-1001; B2 = john.doe@gmail.com; C2 = Ravi-Kumar-Sharma; D2 = India/Karnataka/Bengaluru.

Worked examples and expected outputs

1. Username

```
=TEXTBEFORE("john.doe@gmail.com","@")
```

Expected result: john.doe. Extracts before @.

2. Email domain

```
=TEXTAFTER("john.doe@gmail.com","@")
```

Expected result: gmail.com. Extracts after @.

3. ID prefix

```
=TEXTBEFORE("EMP-2026-1001","-")
```

Expected result: EMP. First occurrence by default.

4. Before second delimiter

```
=TEXTBEFORE("EMP-2026-1001","-",2)
```

Expected result: EMP-2026. Occurrence 2 ends extracted region.

5. After last delimiter

```
=TEXTAFTER("EMP-2026-1001","-",-1)
```

Expected result: 1001. Negative occurrence counts backward.

6. City from location path

```
=TEXTAFTER("India/Karnataka/Bengaluru","/",-1)
```

Expected result: Bengaluru. Extract final segment.

7. Split ID into three cells

```
=TEXTSPLIT("EMP-2026-1001","-")
```

Expected result: EMP | 2026 | 1001. Spills horizontally.

8. Split into rows

```
=TEXTSPLIT("Python,SQL,Excel",,",")
```

Expected result: Python / SQL / Excel (vertical). Omitted column delimiter; comma is row delimiter.

9. Ignore empty parts

```
=TEXTSPLIT("Python,,Excel,SQL",",",,TRUE)
```

Expected result: Python | Excel | SQL. Fourth argument skips empty positions.

10. Two delimiters

```
=TEXTSPLIT("Python,SQL;Excel",{",",";"})
```

Expected result: Python | SQL | Excel. Array constant supplies alternate separators.

11. Missing delimiter handling

```
=TEXTBEFORE("Ravi Kumar","@",1,0,0,"Not Found")
```

Expected result: Not Found. Optional sixth argument handles delimiter absence.

Important concepts and best practices

• Dynamic-array results need empty cells to spill into; blockers cause #SPILL!.

• Missing delimiters ordinarily cause #N/A for TEXTBEFORE/TEXTAFTER.

• Negative instance numbers allow last-occurrence extraction.

• Available in newer Excel versions such as Microsoft 365; older editions may not recognize them.

• TEXTSPLIT does not understand semantic components (such as whether a second name is a middle name).

Common mistakes / limitations

• Do not place TEXTSPLIT where adjacent output cells are populated.

• Split names carefully when some records have two parts and others three.

• Use instance_num rather than character positions when delimiters vary.

LESSON 66 | TEXT and Number/Date Formatting

TEXT returns a formatted text representation of a number, date, time or percent. Use it for readable report labels, fixed-length codes and concatenation; do not confuse it with number formatting that keeps a numeric cell numeric.

Syntax and arguments

TEXT: Format numeric/date/time value as text. Format code is quoted.

```
=TEXT(value,format_text)
```

Example data / worksheet setup

Example table: Employee_ID EMP101; Salary 45678.5; Joining_Date =DATE(2026,1,15); Performance_Score 0.875; Login_Time =TIME(9,5,0).

Worked examples and expected outputs

1. Two decimals

```
=TEXT(45678.5,"0.00")
```

Expected result: 45678.50. Mandatory zeros ensure two decimal places.

2. Thousands and decimals

```
=TEXT(45678.5,"#,##0.00")
```

Expected result: 45,678.50. Grouping separators are included.

3. Display rupee symbol

```
=TEXT(45678.5,"₹#,##0.00")
```

Expected result: ₹45,678.50. Uses three-digit grouping in this example.

4. Readable date

```
=TEXT(DATE(2026,1,15),"dd-mmm-yyyy")
```

Expected result: 15-Jan-2026. Dates are stored internally as serials.

5. Month name

```
=TEXT(DATE(2026,1,15),"mmmm")
```

Expected result: January. Full month name.

6. Day name

```
=TEXT(DATE(2026,1,15),"dddd")
```

Expected result: Thursday. Day of week.

7. Format percent

```
=TEXT(0.875,"0.0%")
```

Expected result: 87.5%. Percentage formatting multiplies displayed number by 100.

8. Time with AM/PM

```
=TEXT(TIME(14,15,0),"hh:mm AM/PM")
```

Expected result: 02:15 PM. 12-hour clock.

9. Pad identifier

```
=TEXT(7,"0000")
```

Expected result: 0007. At least four digits shown.

10. Date in sentence

```
="EMP101 joined on "&TEXT(DATE(2026,1,15),"dd-mmm-yyyy")
```

Expected result: EMP101 joined on 15-Jan-2026. TEXT prevents raw date serial display in concatenation.

Important concepts and best practices

• TEXT results are text. Keep raw numeric values in their original columns for calculations.

• The # digit is optional; 0 is a mandatory digit placeholder.

• Format code m can represent month or minutes depending on context.

• Locale may affect separators, currency display, and language of day/month names.

Common mistakes / limitations

• Keep format codes in double quotes.

• Do not expect numeric operations to always work on formatted text values.

• TEXT is different from Home > Number formatting, which preserves numeric values.

LESSON 67 | TRANSLATE & DETECTLANGUAGE

TRANSLATE converts text between languages using Microsoft Translation Services. DETECTLANGUAGE returns a language code. These service-backed functions are not supported in every Excel edition, may require connectivity, and translations can vary.

Syntax and arguments

TRANSLATE: Translate text to target language using codes such as en, es and fr.

```
=TRANSLATE(text,[source_language],[target_language])
```

DETECTLANGUAGE: Identify the likely language of a text string.

```
=DETECTLANGUAGE(text)
```

Example data / worksheet setup

Example translation data: Good morning / en / es; Thank you / en / fr; Welcome / en / hi; Hello / en / kn. Output wording depends on the service.

Worked examples and expected outputs

1. English to Spanish

```
=TRANSLATE("Good morning","en","es")
```

Expected result: Buenos días (representative). Exact translation may vary by service.

2. English to French

```
=TRANSLATE("Thank you","en","fr")
```

Expected result: Merci (representative). fr means French.

3. Dynamic source and target

```
=TRANSLATE(B2,C2,D2)
```

Expected result: Translation of B2. C2 holds source code; D2 target code.

4. Fix target language cell

```
=TRANSLATE(B2,"en",$G$2)
```

Expected result: Language selected in G2. Absolute reference stays fixed down the column.

5. Auto-detect source

```
=TRANSLATE("Bonjour",,"en")
```

Expected result: Hello (representative). Source argument omitted.

6. Detect language

```
=DETECTLANGUAGE("Bonjour")
```

Expected result: fr (representative). Language detection can be uncertain for very short text.

7. Detect then translate

```
=TRANSLATE(A2,DETECTLANGUAGE(A2),"en")
```

Expected result: English translation. Useful for multilingual feedback.

8. Join translation with label

```
="Spanish: "&TRANSLATE("Good morning","en","es")
```

Expected result: Spanish: Buenos días (representative). Concatenation adds report label.

Important concepts and best practices

• Common codes: en English; es Spanish; fr French; de German; hi Hindi; kn Kannada; ta Tamil; te Telugu.

• Specify source and target explicitly when known for predictable behavior.

• Translation may be throttled or unavailable; #NAME? indicates an unrecognized function in an unsupported edition.

• Text is handled by an online service; check data confidentiality policy before translating sensitive content.

• Review machine translations for high-stakes business use.

Common mistakes / limitations

• Never assume translation outputs will always match a fixed phrase exactly.

• Language detection is uncertain for short or ambiguous expressions.

• Avoid sharing confidential client text without authorization.

• Microsoft 365 feature availability may vary across update channels and platforms.

Quick Reference: All Text Functions

Function	Use	Minimal formula
LEFT	First N characters	=LEFT(A2,3)
RIGHT	Last N characters	=RIGHT(A2,4)
MID	Characters from position	=MID(A2,5,4)
UPPER	Uppercase	=UPPER(A2)
LOWER	Lowercase	=LOWER(A2)
PROPER	Title-style case	=PROPER(A2)
LEN	Character count	=LEN(A2)
TRIM	Remove extra normal spaces	=TRIM(A2)
CLEAN	Remove certain control chars	=CLEAN(A2)
FIND	Locate exact-case match	=FIND("@",A2)
SEARCH	Locate case-insensitive match	=SEARCH("excel",A2)
SUBSTITUTE	Replace matching text	=SUBSTITUTE(A2,"-","_")
REPLACE	Replace by position	=REPLACE(A2,5,4,"2025")
CONCAT	Join text	=CONCAT(A2," ",B2)
TEXTJOIN	Join with delimiter	=TEXTJOIN(" ",TRUE,A2:C2)
TEXTBEFORE	Part before delimiter	=TEXTBEFORE(A2,"@")
TEXTAFTER	Part after delimiter	=TEXTAFTER(A2,"@")
TEXTSPLIT	Split into cells	=TEXTSPLIT(A2,"-")
TEXT	Format as text	=TEXT(A2,"0.00")
TRANSLATE	Translate language	=TRANSLATE(A2,"en","es")
DETECTLANGUAGE	Detect language code	=DETECTLANGUAGE(A2)

Integrated Worked Business Examples

A. Clean a name before standardizing capitalization

```
=PROPER(TRIM(CLEAN(A2)))
```

Example: A2 contains excess spaces and line breaks around a name. CLEAN removes supported control characters, TRIM normalizes regular spaces, and PROPER standardizes capitalization. Verify that removing a line break does not accidentally join words.

B. Build a consistent ID from text and numeric fields

```
=UPPER(B2)&"-"&TEXT(C2,"0000")
```

If B2 is it and C2 is 7, the output is IT-0007. This combines UPPER, TEXT and concatenation.

C. Extract domain and normalize it

```
=LOWER(TEXTAFTER(B2,"@"))
```

If B2 is John.Doe@EXAMPLE.COM, output is example.com. In older Excel, use LOWER(RIGHT(B2,LEN(B2)-FIND("@",B2))).

D. Standardize multi-part names while ignoring blanks

```
=PROPER(TEXTJOIN(" ",TRUE,TRIM(B2),TRIM(C2),TRIM(D2)))
```

With Ravi, a blank middle name, and Kumar, output is Ravi Kumar. The extra TRIM operations clean each component.

E. Create a narrative using date and currency formatting

```
=A2&" joined on "&TEXT(B2,"dd-mmm-yyyy")&" with salary "&TEXT(C2,"₹#,##0.00")
```

If A2 is EMP101, B2 is a real date 15-Jan-2026 and C2 is 45678.5, output: EMP101 joined on 15-Jan-2026 with salary ₹45,678.50.

Final Review: Selecting the Right Function

If your goal is to...	Use...
Take a known number of characters	LEFT / RIGHT / MID
Standardize letter case	UPPER / LOWER / PROPER
Find extra spaces or hidden characters	LEN / TRIM / CLEAN
Find the location of text	FIND / SEARCH
Change a specific word or separator	SUBSTITUTE
Change characters at a known position	REPLACE
Combine name, codes or labels	CONCAT / TEXTJOIN / &
Extract text around a separator	TEXTBEFORE / TEXTAFTER
Split one cell into several cells	TEXTSPLIT
Create a formatted reporting label	TEXT
Translate or identify languages	TRANSLATE / DETECTLANGUAGE

Course note: This handbook includes only worked examples and explanations. It intentionally excludes student practice assignments, quizzes, or graded exercises.